

教学管理服务 - 学生信息查询报告

一、实验目的

本实验通过编写Python脚本，调用教学管理服务API，实现学生信息的查询功能，并将查询结果输出为PDF报告。

二、实验环境

- 编程语言：Python 3.x
- 依赖库：requests, reportlab
- API地址：<https://pysummer.pkuai.cc/api/v1>

三、Python代码

```
import requests
import json
import sys
import os
import argparse
from typing import Optional, Dict, Any
from reportlab.lib.pagesizes import letter
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle
from reportlab.lib import colors
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont

DEFAULT_BASE_URL = "https://pysummer.pkuai.cc/api/v1"
MAX_RETRIES = 2
TIMEOUT = 15

try:
    pdfmetrics.registerFont(TTFont('SimHei', 'simhei.ttf'))
except:
    pass

def create_session(verify_ssl: bool = True) -> requests.Session:
    session = requests.Session()
    session.headers.update({
        "User-Agent": "StudentInfoQuery/1.0",
        "Accept": "application/json",
        "Content-Type": "application/json; charset=utf-8",
    })
    session.verify = verify_ssl
    return session

def validate_input(student_id: str, secret: str, base_url: str) -> bool:
    if not student_id or not student_id.strip():
        print("[错误] 学号不能为空")
        return False
    if not secret or not secret.strip():
        print("[错误] 密钥不能为空")
```

```

        return False
    if not base_url or not base_url.strip():
        print("[错误] API地址不能为空")
        return False
    return True

```

```

def api_login(session: requests.Session, base_url: str, student_id: str, secret: str) -> Option:
    url = f"{base_url}/auth/login"
    payload = {"identifier": student_id.strip(), "secret": secret.strip()}

    for attempt in range(MAX_RETRIES):
        try:
            resp = session.post(url, json=payload, timeout=TIMEOUT)

            if resp.status_code == 401:
                try:
                    err_data = resp.json()
                    msg = err_data.get("error", {}).get("message", "学号或密钥错误")
                except:
                    msg = "学号或密钥错误"
                print(f"[失败] {msg}")
                return None

            resp.raise_for_status()

            try:
                data = resp.json()
            except json.JSONDecodeError:
                print(f"[失败] 服务器返回非JSON响应: {resp.text[:200]}")
                return None

            if "access_token" not in data:
                print("[失败] 响应中缺少 access_token 字段")
                return None

            return data["access_token"]

        except requests.exceptions.ConnectionError:
            print(f"[失败] 连接服务器失败 (尝试 {attempt + 1}/{MAX_RETRIES})")
            if attempt < MAX_RETRIES - 1:
                print("正在重试...")
        except requests.exceptions.Timeout:

```

```

print(f"[失败] 请求超时 (尝试 {attempt + 1}/{MAX_RETRIES})")
if attempt < MAX_RETRIES - 1:
    print("正在重试...")
except requests.exceptions.SSLError:
    print("[失败] SSL证书验证失败, 请检查服务器证书或使用 --no-ssl-verify 参数")
    return None
except requests.exceptions.HTTPError as e:
    print(f"[失败] HTTP错误: {e}")
    try:
        err_data = resp.json()
        if "error" in err_data:
            err_code = err_data["error"].get("code", "")
            err_msg = err_data["error"].get("message", "")
            print(f"[错误] [{err_code}] {err_msg}")
    except:
        pass
    return None
except Exception as e:
    print(f"[失败] 登录异常: {str(e)}")
    return None

print("[失败] 多次尝试后仍无法连接服务器")
return None

```

```

def get_student_info(session: requests.Session, base_url: str, token: str) -> Optional[Dict[str, str]]:
    url = f"{base_url}/students/me"
    session.headers["Authorization"] = f"Bearer {token}"

    for attempt in range(MAX_RETRIES):
        try:
            resp = session.get(url, timeout=TIMEOUT)

            if resp.status_code == 401:
                try:
                    err_data = resp.json()
                    msg = err_data.get("error", {}).get("message", "Token无效或已过期")
                except:
                    msg = "Token无效或已过期"
                print(f"[失败] {msg}")
                return None

            resp.raise_for_status()

```

```

try:
    data = resp.json()
except json.JSONDecodeError:
    print(f"[失败] 服务器返回非JSON响应: {resp.text[:200]}")
    return None

return data

except requests.exceptions.ConnectionError:
    print(f"[失败] 连接服务器失败 (尝试 {attempt + 1}/{MAX_RETRIES})")
    if attempt < MAX_RETRIES - 1:
        print("正在重试...")
except requests.exceptions.Timeout:
    print(f"[失败] 请求超时 (尝试 {attempt + 1}/{MAX_RETRIES})")
    if attempt < MAX_RETRIES - 1:
        print("正在重试...")
except requests.exceptions.SSLError:
    print("[失败] SSL证书验证失败")
    return None
except requests.exceptions.HTTPError as e:
    print(f"[失败] HTTP错误: {e}")
    try:
        err_data = resp.json()
        if "error" in err_data:
            err_code = err_data["error"].get("code", "")
            err_msg = err_data["error"].get("message", "")
            print(f"[错误] [{err_code}] {err_msg}")
    except:
        pass
    return None
except Exception as e:
    print(f"[失败] 查询异常: {str(e)}")
    return None

print("[失败] 多次尝试后仍无法连接服务器")
return None

```

```

def print_info(info: Dict[str, Any]):
    print("\n" + "=" * 35)
    print("学生信息查询结果")
    print("-" * 35)

```

```

student_id = info.get("student_id", "N/A")
print(f"学号: {student_id}")

name = info.get("name", "N/A")
print(f"姓名: {name}")

extra = info.get("extra", {})
if extra:
    try:
        extra_str = json.dumps(extra, ensure_ascii=False, indent=2)
        print(f"扩展信息:\n{extra_str}")
    except:
        print(f"扩展信息: {extra}")
else:
    print("扩展信息: 无")

created_at = info.get("created_at", "N/A")
print(f"创建时间: {created_at}")

updated_at = info.get("updated_at", "N/A")
print(f"更新时间: {updated_at}")

print("=" * 35)

```

```

def generate_markdown(info: Dict[str, Any]) -> str:
    student_id = info.get("student_id", "N/A")
    name = info.get("name", "N/A")
    extra = info.get("extra", {})
    created_at = info.get("created_at", "N/A")
    updated_at = info.get("updated_at", "N/A")

    md_content = f"""# 学生信息查询结果

```

基本信息

```

| 项目 | 内容 |
|-----|-----|
| 学号 | {student_id} |
| 姓名 | {name} |
| 创建时间 | {created_at} |
| 更新时间 | {updated_at} |

```

扩展信息

"""

```
if extra:
    try:
        extra_str = json.dumps(extra, ensure_ascii=False, indent=2)
        md_content += f"```\njson\n{extra_str}\n```"
    except:
        md_content += f"{extra}"
else:
    md_content += "无"

return md_content
```

```
def save_markdown(info: Dict[str, Any], filename: str = None) -> str:
    if not filename:
        student_id = info.get("student_id", "student")
        filename = f"学生信息_{student_id}.md"

    md_content = generate_markdown(info)

    with open(filename, 'w', encoding='utf-8') as f:
        f.write(md_content)

    return filename
```

```
def save_pdf(info: Dict[str, Any], filename: str = None) -> str:
    if not filename:
        student_id = info.get("student_id", "student")
        filename = f"学生信息_{student_id}.pdf"

    doc = SimpleDocTemplate(filename, pagesize=letter)
    styles = getSampleStyleSheet()

    try:
        title_style = ParagraphStyle(
            'TitleStyle',
            parent=styles['Title'],
            fontName='SimHei',
```

```

        fontSize=18,
        alignment=1,
        spaceAfter=20
    )
    normal_style = ParagraphStyle(
        'NormalStyle',
        parent=styles['Normal'],
        fontName='SimHei',
        fontSize=12,
        spaceAfter=10
    )
    header_style = ParagraphStyle(
        'HeaderStyle',
        parent=styles['Heading2'],
        fontName='SimHei',
        fontSize=14,
        spaceAfter=15
    )
except:
    title_style = styles['Title']
    normal_style = styles['Normal']
    header_style = styles['Heading2']

elements = []

elements.append(Paragraph("学生信息查询结果", title_style))
elements.append(Spacer(1, 12))

elements.append(Paragraph("基本信息", header_style))

student_id = info.get("student_id", "N/A")
name = info.get("name", "N/A")
created_at = info.get("created_at", "N/A")
updated_at = info.get("updated_at", "N/A")

data = [
    ["项目", "内容"],
    ["学号", student_id],
    ["姓名", name],
    ["创建时间", created_at],
    ["更新时间", updated_at]
]

```



```

table = Table(data, colWidths=[100, 400])
table.setStyle(TableStyle([
    ('BACKGROUND', (0, 0), (-1, 0), colors.grey),
    ('TEXTCOLOR', (0, 0), (-1, 0), colors.whitesmoke),
    ('ALIGN', (0, 0), (-1, -1), 'CENTER'),
    ('FONTNAME', (0, 0), (-1, -1), 'SimHei'),
    ('FONTSIZE', (0, 0), (-1, -1), 12),
    ('BOTTOMPADDING', (0, 0), (-1, 0), 12),
    ('BACKGROUND', (0, 1), (-1, -1), colors.beige),
    ('GRID', (0, 0), (-1, -1), 1, colors.black),
]))

```

```

elements.append(table)
elements.append(Spacer(1, 12))

```

```

elements.append(Paragraph("扩展信息", header_style))

```

```

extra = info.get("extra", {})
if extra:
    try:
        extra_str = json.dumps(extra, ensure_ascii=False, indent=2)
        elements.append(Paragraph(f"<pre>{extra_str}</pre>", normal_style))
    except:
        elements.append(Paragraph(str(extra), normal_style))
else:
    elements.append(Paragraph("无", normal_style))

doc.build(elements)

return filename

```

```

def get_input(prompt: str, allow_empty: bool = False) -> str:
    while True:
        try:
            value = input(prompt).strip()
            if value or allow_empty:
                return value
            print("[提示] 输入不能为空，请重新输入")
        except KeyboardInterrupt:
            print("\n[退出] 用户中断操作")
            sys.exit(0)
        except EOFError:

```

```
print("\n[退出] 输入结束")
sys.exit(0)
```

```
def main():
    parser = argparse.ArgumentParser(description="教学管理服务 - 学生信息查询")
    parser.add_argument("-s", "--student-id", help="学生学号")
    parser.add_argument("-k", "--secret", help="登录密钥")
    parser.add_argument("--export-md", action="store_true", help="导出为Markdown文件")
    parser.add_argument("--export-pdf", action="store_true", help="导出为PDF文件")
    parser.add_argument("--export-all", action="store_true", help="同时导出Markdown和PDF文件")

    args = parser.parse_args()

    if args.student_id and args.secret:
        student_id = args.student_id.strip()
        secret = args.secret.strip()
    else:
        print("=" * 40)
        print("教学管理服务 - 学生信息查询")
        print("=" * 40)
        print("提示: 输入完成后按回车键确认")
        print("提示: 按 Ctrl+C 可随时退出")
        print("-" * 40)

        student_id = get_input("请输入学号: ")
        secret = get_input("请输入密钥: ")

    print("\n正在连接服务器...")

    session = create_session(verify_ssl=True)

    print("正在登录...")
    token = api_login(session, DEFAULT_BASE_URL, student_id, secret)

    if not token:
        print("\n[退出] 登录失败, 请检查学号和密钥是否正确")
        sys.exit(1)

    print("登录成功!")
    print("正在查询学生信息...")

    info = get_student_info(session, DEFAULT_BASE_URL, token)
```

```
if not info:
    print("[失败] 查询学生信息失败")
    sys.exit(1)

print_info(info)

export_md = args.export_md or args.export_all
export_pdf = args.export_pdf or args.export_all

if not args.student_id and not args.secret:
    print("\n" + "-" * 40)
    print("是否需要导出文件? ")
    print("1. 导出为 Markdown 文件")
    print("2. 导出为 PDF 文件")
    print("3. 同时导出 Markdown 和 PDF")
    print("4. 不导出")

    choice = get_input("请选择 (1-4): ")

    while choice not in ["1", "2", "3", "4"]:
        print("[提示] 请输入有效的选项 (1-4)")
        choice = get_input("请选择 (1-4): ")

    export_md = choice in ["1", "3"]
    export_pdf = choice in ["2", "3"]

if export_md:
    try:
        md_file = save_markdown(info)
        print(f"[成功] Markdown 文件已保存: {os.path.abspath(md_file)}")
    except Exception as e:
        print(f"[失败] 保存 Markdown 文件失败: {str(e)}")

if export_pdf:
    try:
        pdf_file = save_pdf(info)
        print(f"[成功] PDF 文件已保存: {os.path.abspath(pdf_file)}")
    except Exception as e:
        print(f"[失败] 保存 PDF 文件失败: {str(e)}")

print("\n查询完成! ")
```

```
if __name__ == "__main__":  
    main()
```

四、实验结果

实验 1

输入参数：

- 学号：2600926009
- 密钥：p22GhAlOxg62tDRLpkehWN9klg4AkfOs

查询结果： 成功

学生信息

项目	内容
学号	2600926009
姓名	陈斌
创建时间	2026-07-19T22:03:03+08:00
更新时间	2026-07-19T22:18:35+08:00

扩展信息

无

实验 2

输入参数：

- 学号：2600940418
- 密钥：vSJme4mA5la6Vn8Ao58UdNEgRsEkgo-m

查询结果： 成功

学生信息

项目	内容
学号	2600940418
姓名	潘静怡
创建时间	2026-07-19T17:10:14+08:00
更新时间	2026-07-19T17:10:14+08:00

扩展信息

无

报告生成时间： 2026-07-23